

Exercise1:

Scenario1:

```
BEGIN
  FOR c IN (SELECT CustomerID FROM Customers)
  LOOP
    IF MONTHS_BETWEEN(SYSDATE,
      (SELECT DOB FROM Customers WHERE CustomerID=c.CustomerID))/12 > 60
    THEN
      UPDATE Loans
      SET InterestRate = InterestRate - 1
      WHERE CustomerID=c.CustomerID;
    END IF;
  END LOOP;

  COMMIT;
END;
/
```

Scenario2:

```
ALTER TABLE Customers ADD IsVIP VARCHAR2(10);

BEGIN
  FOR c IN (SELECT CustomerID,Balance FROM Customers)
  LOOP
    IF c.Balance > 10000 THEN
      UPDATE Customers
      SET IsVIP='TRUE'
      WHERE CustomerID=c.CustomerID;
    ELSE
```

```
UPDATE Customers
SET IsVIP='FALSE'
WHERE CustomerID=c.CustomerID;

END IF;

END LOOP;

COMMIT;

END;

/
```

Scenario 3:

```
BEGIN

FOR l IN
(
SELECT c.Name,l.EndDate
FROM Customers c
JOIN Loans l
ON c.CustomerID=l.CustomerID
WHERE l.EndDate BETWEEN SYSDATE AND SYSDATE+30
)
LOOP
DBMS_OUTPUT.PUT_LINE(
'Reminder: Loan due for '||l.Name);
END LOOP;

END;

/
```

Exercise 2:

Scenario1:

CREATE OR REPLACE PROCEDURE SafeTransferFunds

(

fromAcc NUMBER,

toAcc NUMBER,

amount NUMBER

)

AS

balance NUMBER;

BEGIN

SELECT Balance INTO balance

FROM Accounts

WHERE AccountID=fromAcc;

IF balance < amount THEN

RAISE_APPLICATION_ERROR(-20001,

'Insufficient Balance');

END IF;

UPDATE Accounts

SET Balance=Balance-amount

WHERE AccountID=fromAcc;

UPDATE Accounts

SET Balance=Balance+amount

WHERE AccountID=toAcc;

COMMIT;

```
EXCEPTION
WHEN OTHERS THEN
ROLLBACK;
DBMS_OUTPUT.PUT_LINE(SQLERRM);
END;
/
```

Scenario2:

```
CREATE OR REPLACE PROCEDURE UpdateSalary
(
  id NUMBER,
  percent NUMBER
)
AS
BEGIN
  UPDATE Employees
  SET Salary = Salary + (Salary*percent/100)
  WHERE EmployeeID=id;
  IF SQL%ROWCOUNT=0 THEN
    RAISE_APPLICATION_ERROR(-20002,
    'Employee not found');
  END IF;
  COMMIT;
EXCEPTION
WHEN OTHERS THEN
  DBMS_OUTPUT.PUT_LINE(SQLERRM);
END;
/
```

Scenario3:

```
CREATE OR REPLACE PROCEDURE AddNewCustomer
```

```
(
```

```
id NUMBER,
```

```
name VARCHAR2,
```

```
bal NUMBER
```

```
)
```

```
AS
```

```
BEGIN
```

```
INSERT INTO Customers
```

```
VALUES(id,name,SYSDATE,bal,SYSDATE);
```

```
COMMIT;
```

```
EXCEPTION
```

```
WHEN DUP_VAL_ON_INDEX THEN
```

```
DBMS_OUTPUT.PUT_LINE(
```

```
'Customer already exists');
```

```
END;
```

```
/
```

Exercise3:

Scenario1:

```
CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest
```

```
AS
```

```
BEGIN
```

```
UPDATE Accounts
```

```
SET Balance = Balance + (Balance * 0.01)
```

```
WHERE AccountType='Savings';
```

```
COMMIT;
```

```
END;
```

```
/
```

Scenario2:

```
CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus
```

```
(
```

```
dept VARCHAR2,
```

```
bonus NUMBER
```

```
)
```

```
AS
```

```
BEGIN
```

```
UPDATE Employees
```

```
SET Salary = Salary + (Salary*bonus/100)
```

```
WHERE Department=dept;
```

```
COMMIT;
```

```
END;
```

```
/
```

Scenario3:

```
CREATE OR REPLACE PROCEDURE TransferFunds
```

```
(
```

```
fromAcc NUMBER,
```

```
toAcc NUMBER,
```

```
amount NUMBER
```

```
)
```

```
AS
```

```
bal NUMBER;
```

```
BEGIN
```

```
SELECT Balance INTO bal
```

```

FROM Accounts
WHERE AccountID=fromAcc;
IF bal < amount THEN
RAISE_APPLICATION_ERROR(
-20001,'Insufficient balance');
END IF;
UPDATE Accounts
SET Balance=Balance-amount
WHERE AccountID=fromAcc;
UPDATE Accounts
SET Balance=Balance+amount
WHERE AccountID=toAcc;
COMMIT;
END;
/

```

Exercise4:

Scenario1:

```

CREATE OR REPLACE FUNCTION CalculateAge
(
dob DATE
)
RETURN NUMBER
AS
age NUMBER;
BEGIN
age :=
TRUNC(MONTHS_BETWEEN(SYSDATE,dob)/12);

```

```
RETURN age;
```

```
END;
```

```
/
```

Scenario2:

```
CREATE OR REPLACE FUNCTION CalculateMonthlyInstallment
```

```
(
```

```
amount NUMBER,
```

```
rate NUMBER,
```

```
years NUMBER
```

```
)
```

```
RETURN NUMBER
```

```
AS
```

```
emi NUMBER;
```

```
BEGIN
```

```
emi :=
```

```
(amount + amount*(rate/100)*years)/(years*12);
```

```
RETURN emi;
```

```
END;
```

```
/
```

Scenario3:

```
CREATE OR REPLACE FUNCTION HasSufficientBalance
```

```
(
```

```
id NUMBER,
```

```
amount NUMBER
```

```
)
```

```
RETURN BOOLEAN
```

```
AS
```



```
bal NUMBER;  
  
BEGIN  
  
SELECT Balance INTO bal  
  
FROM Accounts  
  
WHERE AccountID=id;  
  
IF bal>=amount THEN  
  
RETURN TRUE;  
  
ELSE  
  
RETURN FALSE;  
  
END IF;  
  
END;  
  
/
```

Exercise5:

Scenario1:

```
CREATE OR REPLACE TRIGGER UpdateCustomerLastModified  
  
BEFORE UPDATE ON Customers  
  
FOR EACH ROW  
  
BEGIN  
  
:NEW.LastModified := SYSDATE;  
  
END;  
  
/
```

Scenario2:

```
CREATE TABLE AuditLog  
  
(  
  
TransactionID NUMBER,  
  
LogDate DATE  
  
);
```

```
CREATE OR REPLACE TRIGGER LogTransaction
AFTER INSERT ON Transactions
FOR EACH ROW
BEGIN
INSERT INTO AuditLog
VALUES(:NEW.TransactionID,SYSDATE);
END;
/
```

Scenario3:

```
CREATE OR REPLACE TRIGGER CheckTransactionRules
BEFORE INSERT ON Transactions
FOR EACH ROW
DECLARE
bal NUMBER;
BEGIN
SELECT Balance INTO bal
FROM Accounts
WHERE AccountID=:NEW.AccountID;
IF :NEW.TransactionType='Withdrawal'
AND :NEW.Amount>bal THEN
RAISE_APPLICATION_ERROR(
-20001,'Insufficient balance');
END IF;
IF :NEW.TransactionType='Deposit'
AND :NEW.Amount<=0 THEN
RAISE_APPLICATION_ERROR(
-20002,'Invalid deposit');
END IF;
```

END;

/

Exercise6:

Scenario1:

DECLARE

CURSOR GenerateMonthlyStatements IS

SELECT AccountID,Amount,TransactionType

FROM Transactions

WHERE EXTRACT(MONTH FROM TransactionDate)

=

EXTRACT(MONTH FROM SYSDATE);

BEGIN

FOR t IN GenerateMonthlyStatements

LOOP

DBMS_OUTPUT.PUT_LINE(

'Account: '||t.AccountID||

' Amount: '||t.Amount||

' Type: '||t.TransactionType);

END LOOP;

END;

/

Scenario2:

DECLARE

CURSOR ApplyAnnualFee IS

SELECT AccountID

FROM Accounts;

```
fee NUMBER := 500;

BEGIN

FOR a IN ApplyAnnualFee
LOOP

UPDATE Accounts

SET Balance = Balance - fee

WHERE AccountID=a.AccountID;

END LOOP;

COMMIT;

END;

/
```

Scenario3:

```
DECLARE

CURSOR UpdateLoanInterestRates IS

SELECT LoanID,InterestRate

FROM Loans;

BEGIN

FOR l IN UpdateLoanInterestRates
LOOP

UPDATE Loans

SET InterestRate = InterestRate + 1

WHERE LoanID=l.LoanID;

END LOOP;

COMMIT;

END;

/
```

Exercise7:

Scenario1:

CREATE OR REPLACE PACKAGE CustomerManagement

AS

PROCEDURE AddCustomer

(

id NUMBER,

name VARCHAR2,

balance NUMBER

);

PROCEDURE UpdateCustomer

(

id NUMBER,

balance NUMBER

);

FUNCTION GetBalance

(

id NUMBER

)

RETURN NUMBER;

END CustomerManagement;

/

Scenario2:

CREATE OR REPLACE PACKAGE EmployeeManagement

AS

PROCEDURE HireEmployee

(

```
id NUMBER,
name VARCHAR2,
salary NUMBER
);
PROCEDURE UpdateEmployee
(
id NUMBER,
salary NUMBER
);
FUNCTION AnnualSalary
(
id NUMBER
)
RETURN NUMBER;
END EmployeeManagement;
/
CREATE OR REPLACE PACKAGE BODY EmployeeManagement
AS
PROCEDURE HireEmployee
(
id NUMBER,
name VARCHAR2,
salary NUMBER
)
AS
BEGIN
INSERT INTO Employees
(EmployeeID,Name,Salary,HireDate)
```

```

VALUES
(id,name,salary,SYSDATE);
END;
PROCEDURE UpdateEmployee
(
id NUMBER,
salary NUMBER
)
AS
BEGIN
UPDATE Employees
SET Salary=salary
WHERE EmployeeID=id;
END;
FUNCTION AnnualSalary
(
id NUMBER
)
RETURN NUMBER
AS
sal NUMBER;
BEGIN
SELECT Salary INTO sal
FROM Employees
WHERE EmployeeID=id;
RETURN sal*12;
END;
END EmployeeManagement;

```

/

Scenario3:

CREATE OR REPLACE PACKAGE AccountOperations

AS

PROCEDURE OpenAccount

(

id NUMBER,

customer NUMBER,

balance NUMBER

);

PROCEDURE CloseAccount

(

id NUMBER

);

FUNCTION TotalBalance

(

customer NUMBER

)

RETURN NUMBER;

END AccountOperations;

/

CREATE OR REPLACE PACKAGE BODY AccountOperations

AS

PROCEDURE OpenAccount

(

id NUMBER,

customer NUMBER,

balance NUMBER


```
)  
AS  
BEGIN  
INSERT INTO Accounts  
VALUES(id,customer,'Savings',  
balance,SYSDATE);  
END;  
PROCEDURE CloseAccount  
(  
id NUMBER  
)  
AS  
BEGIN  
DELETE FROM Accounts  
WHERE AccountID=id;  
END;  
FUNCTION TotalBalance  
(  
customer NUMBER  
)  
RETURN NUMBER  
AS  
total NUMBER;  
BEGIN  
SELECT SUM(Balance)  
INTO total  
FROM Accounts  
WHERE CustomerID=customer;
```

RETURN total;

END;

END AccountOperations;

/